

```
cout << ans << '\n';
return 0;
```

在整除分块中，向下取整分块是最常见的。此外，还有向上取整分块、取模转换为整除、取模分块等多种类型。由于本书篇幅有限，这些内容留给读者自行探索。相关的例题可以参考

Coding平台：

- StarryCoding P135 【模板】整除分块（向下取整）。
- StarryCoding P138 【模板】整除分块（向上取整）。
- StarryCoding P139 【模板】整除分块（取模）。
- StarryCoding P140 【模板】整除分块（二维）。

2.3 离散化

离散化是一种编程技巧，旨在将无限空间中的有限个体映射到有限空间中，以提高算法效率。简单来说，离散化通过在不改变数据相对大小的前提下，缩小数据的表示范围。例如，将数据{1, 999, 100000, 15}处理为{1, 3, 4, 2}。这种方法尤其适用于序列值域较大时，可以降低算法的复杂度，改进效率低下的算法，甚至使某些原本无法实现的算法变得可行。例如，在构建线段树时，离散化是必不可少的，如图12-5所示。

[10, 23, 35, 3, -40, 3]



[3, 4, 5, 2, 1, 2]

图 12-5 构造的线段树

可以看出，离散化后的数据，它们之间相对大小的关系保持不变。下面是一个简单的离散化示例代码：

```
#include<bits/stdc++.h>
using namespace std;
```

```
const int MAXN = 200000+10; // 定义常量，表示数组最大长度
int a[MAXN]; // 存储输入的数组
int b[MAXN]; // 存储排序后的数组，用于离散化
```



```

int main() {

    int n;    // 数组长度
    cin >> n;
    for(int i = 0; i < n; ++i) {
        cin >> a[i];
        b[i] = a[i]; // 复制原数组，用于排序
    }

    // 离散化过程
    sort(b, b + n); // 对b数组进行排序
    int len = unique(b, b + n) - b; // 调用unique去重，并返回去重后数组的长度

    for(int i = 0; i < n; ++i) {
        // lower_bound返回第一个不小于a[i]的元素的迭代器位置
        // 迭代器减去b的起始地址得到的就是离散化后的值
        // 1 2 2 4 5 lower_bound(, , 2);
        a[i] = lower_bound(b, b + len, a[i]) - b;
        // a[i] = upper_bound(b, b + len, a[i]) - b;
        cout << a[i] << ' '; // 输出离散化后的值
    }
    cout << '\n';

    return 0;
}

```

例题讲解

【例】StarryCoding P63 我的数组很长，你算一下

题目描述

小e有一个很长的数组，数组的下标范围为 $[0, 10^9]$ ，初始时每个元素均为0。

现在，他想进行 n 次操作，每次操作将某个下标 i 处的元素加上 x 。

然后，他给出了 q 次询问，每次询问的格式为 l, r ，你需要回答下标在区间 $[l, r]$ 内所有元素的和。

输入格式

第一行：两个整数 n 和 q ($1 \leq n, q \leq 10^5$)。

接下来 n 行：每行两个整数 i 和 x ($0 \leq i \leq 10^9, 0 \leq x \leq 10^4$)。

再接下来 q 行：每行两个整数 l 和 r ($0 \leq l \leq r \leq 10^9$)。

输出格式

输出共 q 行，每行对应一个询问的答案，即区间 $[l, r]$ 内所有元素的和。

样例输入

```
4 2
0 2
3 3
1 5
0 1
0 1
1 3
```

样例输出

```
8
8
```

解题思路

将所有操作中涉及的下标记录为关键点，忽略未被操作的下标。通过离散化处理，将这些关键点映射到一个连续的索引范围，从而将问题转化为一个简单的前缀和问题。

为了方便实现，我们使用vector动态数组来存储离散化后的数据。

```
#include <bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;    // 定义数组最大长度，至多有 3e5 个关键点
using ll = long long;     // 简化长整型名称
vector<ll> X;              // 存储所有需要离散化的点

int bin(ll x){
    // lower_bound 找到 X 中第一个不小于 x 的位置，返回索引 +1
    return lower_bound(X.begin(), X.end(), x) - X.begin() + 1;
}

ll a[N], prefix[N];       // a 数组存储离散化后的点的值变化量，prefix 为前缀和数组

int main()
{
    // 加速输入/输出流，提高运行效率
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);

    // 输入操作次数 n 和询问次数 q
    int n, q; cin >> n >> q;
    vector<pair<ll, ll> > add, que;    // add 存储增加操作，que 存储查询操作

    for(int i = 1; i <= n; ++ i){      // 处理 n 次增加操作
        ll x, y; cin >> x >> y;      // x 是下标，y 是增加的值
        add.push_back({x, y});        // 保存操作到 add 数组
        X.push_back(x);               // 将下标 x 加入离散化数组 X
    }

    while(q --)                      // 处理 q 次区间查询
```



```

    ll l, r; cin >> l >> r;          // l 和 r 是查询的区间范围
    que.push_back({l, r});           // 保存查询到 que 数组
    X.push_back(l), X.push_back(r); // 将区间端点 l 和 r 加入离散化数组 x
}

// 离散化操作
sort(X.begin(), X.end());           // 对x排序
X.erase(unique(X.begin(), X.end()), X.end()); // 去除重复元素, 得到离散化后的关键点
// 处理增加操作

for(const auto &[x, y] : add)
    a[bin(x)] += y; // 将 x 离散化为索引, 并将值 y 加到对应位置

// 构建前缀和数组
for(int i = 1; i <= 3e5; ++ i)
    prefix[i] = prefix[i - 1] + a[i]; // 前缀和: prefix[i] 表示 [1, i] 区间的累加值

// 处理查询操作
for(const auto &[l, r] : que){
    // 通过前缀和快速计算区间 [l, r] 的和
    cout << prefix[bin(r)] - prefix[bin(l) - 1] << '\n';
}
return 0;
}

```

12.4 离线思想

在算法竞赛中, “离线”和“在线”的概念与我们日常生活中的理解不同, 它并不是指“是否联网”, 而是指对于题目的询问, 是否需要按顺序逐个回答, 或者可以在获取所有询问后, 重新排序并以特殊的顺序计算结果, 最后统一输出答案。

离线处理是指将题目的所有询问保存下来, 通过特定的方式分别计算答案, 最后按照询问的原本顺序输出结果。采用离线处理的前提是, 题目的各项询问相互独立。

与之相反, “强制在线”意味着在接收到一个新询问前, 必须先算出当前询问的答案。

虽然大多数题目是在线的, 但离线思想可以帮助我们优化时间复杂度, 设计出更高效的算法。

简单的离线思想通常基于题目询问的单调性, 即通过重新排列询问顺序, 尽量复用计算过程, 而优化时间复杂度。接下来介绍的莫队算法和CDQ分治都是基于离线思想的经典算法。

12.5 莫队算法

莫队算法主要用于离线处理通常只包含查询、不涉及修改的区间问题。普通的莫队算法只能